Does Bellman-Residual Feedback Help Adaptive State Aggregation? A Controlled Negative Result

Sunny Zhang
University of Toronto
ssunny.zhang@mail.utoronto.ca

Abstract

Adaptive state aggregation replaces full-state Bellman updates with updates on groups of states having similar values. A natural refinement rule shrinks the group width in response to the Bellman residual, but any gain may come from annealing itself rather than from feedback. I isolate these effects on a 9,261-state inventory-control MDP by comparing $\varepsilon_t = \max\{0.05, 0.084 \ \mathrm{span}(TV_t - V_t)\}$ with a geometric timetable matched in initial width, floor, and decay rate. Across three global/aggregate update mixes and 20 paired sampling seeds, every 95% bootstrap interval for the final residual-minus-geometric value error lies inside a preregistered ± 0.02 practical-equivalence region. Residual feedback is never practically better; in the global-heavy mix it is slightly worse ($+0.00632$), although policy-loss rankings disagree with value-error rankings. Full-state value iteration also reaches the ground-truth stopping tolerance in fewer charged backups. This case study suggests that the useful ingredient is a refinement timetable, not residual feedback, and illustrates why adaptive dynamic programming should be evaluated against open-loop and decision-quality controls.

1 Introduction

State aggregation combats the state-space cost of dynamic programming by solving a smaller surrogate MDP [1–3]. In the adaptive scheme of Chen et al. [4], states with nearby current values share a group, and ordinary Bellman sweeps alternate with stochastic group sweeps. A fixed bin width ε creates a familiar cost–accuracy tradeoff: coarse groups are cheap but impose approximation error, while fine groups preserve accuracy but perform more backups.

This invites a feedback controller. When the Bellman residual is large, a coarse representation may suffice; as the residual contracts, the algorithm can refine. The key identification problem is that feedback also creates an annealing schedule. Comparing it only with a fixed width cannot tell whether the residual signal helps or whether almost any schedule with the same decay would work.

I test the signal itself. A residual-driven rule is compared with an open-loop geometric control matched in starting width, floor, and approximate time to the floor. The original experiment was preregistered at one update mix and a 20 ms endpoint. I preserve that result, including the endpoint's failure, and add a work-indexed robustness check at three update mixes. The result is negative but operationally useful: residual feedback provides no practical improvement over the matched timetable on this instance. Exact rankings instead depend on update allocation and on whether performance is measured by value error or decision loss.

2 Related work

Aggregation and adaptive resolution. Classical hard aggregation replaces each block of states by a common value; approximation and induced-policy losses can be related to within-block value variation [1, 5], while state-abstraction theory separates value-, action-, reward-, and transition-based notions of similarity [3]. Several methods adapt the representation during computation. Bertsekas and Castanon [2] group states by current residual magnitude when accelerating policy evaluation, and Singh et al. [6] adapt soft cluster memberships to reduce squared Bellman error. Variable-resolution control instead splits cells using value, policy, influence, or variance criteria [7]; recent progressive disaggregation similarly slices regions relative to the evolving value function [8]. I use the simpler construction of Chen et al. [4], which repeatedly bins states by their current values and has an asymptotic error guarantee of $2\varepsilon/(1-\gamma)$ when within-bin value variation is at most ε.

Bellman residuals and computational effort. Bellman residuals provide computable value and greedy-policy error certificates [9]; their span seminorm has also been optimized directly in approximate value iteration [10]. Residual-based prioritized sweeping and related solvers use local errors to decide which states or partitions to back up [11, 12]. Yet a small residual need not be a reliable proxy for the decision objective under approximation [13], motivating my separate policy-loss evaluation. My use of the residual is narrower: its global span changes one resolution parameter, not group membership, state priority, or the fitting objective. I therefore do not propose a new abstraction or convergence theorem; I test whether that feedback improves the finite-compute trajectory after controlling for the open-loop annealing path it induces.

3 Method

For a finite discounted cost MDP, define

$$(TV)(s) = \min_a \left[c(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \right]. \tag{1}$$

The solver bins $V(s)$ into occupied intervals of width ε. A global iteration evaluates (1) for every state. An aggregate iteration samples one state per group, performs one lifted Bellman backup for that representative, and updates the group value with step size $\alpha_t = 1/\sqrt{t}$. A schedule (ℓ_g, ℓ_a) repeats ℓ_g global and ℓ_a aggregate iterations. Rebinning occurs on entry to each aggregate phase, and lifting occurs on return to the global phase.

After each global sweep I record $r_t = \operatorname{span}(TV_t - V_t) = \max_s (TV_t - V_t)(s) - \min_s (TV_t - V_t)(s)$. The tested feedback rule is

$$\varepsilon_t^{\text{RESIDUAL}} = \max\{\varepsilon_{\min}, c r_t\}, \qquad c = 0.084, \quad \varepsilon_{\min} = 0.05. \tag{2}$$

Its first finite width is 0.4985 and it reaches the floor after roughly 14 aggregate entries. The main control never reads r_t:

$$\varepsilon_j^{\text{GEOMETRIC}} = \max\left\{ \varepsilon_{\min}, 0.4985 \left(\frac{\varepsilon_{\min}}{0.4985} \right)^{j/13} \right\}. \tag{3}$$

Thus RESIDUAL and fast GEOMETRIC differ primarily in closed-loop versus open-loop refinement. A fixed $\varepsilon = 0.05$ arm tests whether refinement is useful at all. Slower geometric and coarser fixed controls are retained in the artifact but are not needed for the central contrast.

Inventory MDP. A state is the inventory vector $q \in \{-10, \dots, 10\}^3$, giving $|\mathcal{S}| = 21^3 = 9{,}261$. Five joint quote-aggressiveness actions have fill probabilities $(.1, .2, .4, .6, .8)$ and spreads $(1, .8, .5, .3, .15)$. A transition is no fill or a one-unit fill on either side of one asset. The one-period cost combines spread revenue with correlated inventory risk, $c(q,a) = 0.02 q^\top \Sigma q - \delta_a f_a L(q)$, where Σ has unit diagonal and correlation 0.5, and $L(q)$ removes infeasible boundary fills. I use $\gamma = .95$ and rescale costs so $\|V^*\|_\infty = 100$. This is a small market-making-style control model [14, 15], not a calibrated market simulator.

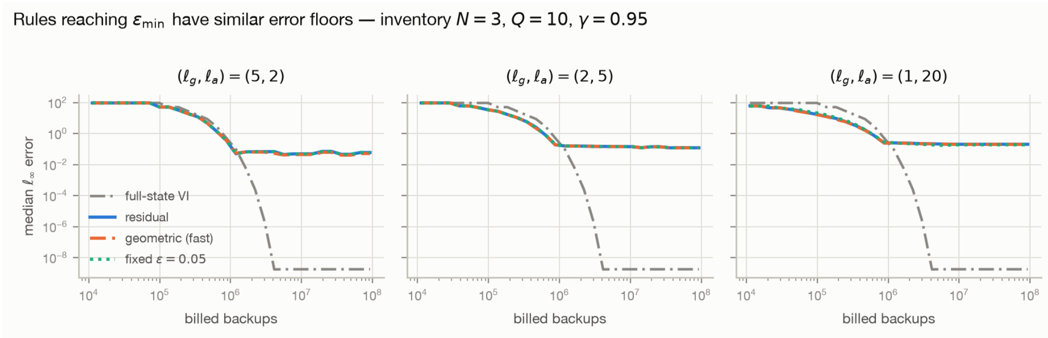

Figure 1: Median value error against billed Bellman backups; terminal estimates are retained after an arm's 10,000-iteration horizon. Curves for residual, fast geometric, and fixed-0.05 nearly coincide within each update mix. The full-state VI trajectory is deterministic and is repeated for reference; its convergence check stops after 454 sweeps (4.20M backups), although the plotted diagnostic continues to the common 10,000-sweep horizon.

Table 1: Final iterate over 20 paired seeds. Columns R/G/F are residual, fast geometric, and fixed $\varepsilon = .05$. The interval is for the median paired error difference R−G. Policy loss is reported in the same order.

Mix	$\mathrm{err}_\infty(\mathrm{R})$	$\mathrm{err}_\infty(\mathrm{G})$	$\mathrm{err}_\infty(\mathrm{F})$	R−G (95% CI)	Policy loss R/G/F
$(5, 2)$	.05001	.04368	.05001	.00632 [.00632,.00633]	.00210/.00278/.00210
$(2, 5)$	.13882	.13882	.13791	$-$.000003 [$-$.000412,.000000]	.01518/.01609/.01280
$(1, 20)$	.20343	.20053	.19093	$-$.00018 [$-$.00208,.00609]	.05178/.05178/.01649

Protocol and metrics. Each arm runs 10,000 iterations at $(5, 2)$, $(2, 5)$, and $(1, 20)$. The MDP is deterministic; seeds 0–19 vary the within-group sampling stream and are paired across arms. The primary reported metric is $\mathrm{err}_\infty = \|V - V^*\|_\infty$. I also evaluate the greedy policy π_V by $\|V^{\pi_V} - V^*\|_\infty$.

Work-indexed curves use billed Bellman backups $B = n_g|\mathcal{S}| + \sum_t K_t$, where K_t is the number of occupied groups. This accounting is favorable to aggregation: rebinning and lifting add $(n_r + n_l)|\mathcal{S}|$ state operations, and the residual scan is not charged. For final paired differences I report the median and a seeded 10,000-resample percentile-bootstrap interval. The preregistered practical-equivalence region is $[-0.02, 0.02]$ on the 100-normalized value scale.

4 Results

Figure 1 shows the central negative result. Once the residual and geometric rules reach $\varepsilon_{\min}$, their work–error trajectories are visually inseparable at the plot scale. Table 1 makes the small differences visible. At all three mixes, the full paired interval for residual minus fast geometric lies inside the ± 0.02 equivalence region. At $(5, 2)$ the interval excludes zero and every seed favors geometric in value error, but the median difference is only 0.00632. At the preregistered $(2, 5)$ mix the median difference is -2.9×10^{-6}, with an interval straddling zero. Residual feedback is therefore never practically better and is sometimes detectably, though not practically, worse.

The fixed control exposes two qualifications. First, at $(1, 20)$ residual is worse than fixed-0.05 by 0.01250 in value error (95% CI $[0.00745, 0.01561]$) and by 0.03529 in policy loss. Second, value accuracy does not order decision quality: at $(5, 2)$ fast geometric has the lowest value error but higher policy loss than residual, while at $(2, 5)$ the slow geometric arm (artifact only) has the lowest value error and 2.5 times fixed-0.05's policy loss. Feedback cannot be justified by either metric here.

The mixes also consume different work. Median residual billed backups are 67.68M, 30.21M, and 9.18M for $(5, 2)$, $(2, 5)$, and $(1, 20)$; maintenance adds 39%, 88%, and 96% respectively. Hence the lower final error of the global-heavy mix is not a causal "schedule wins" result: it purchases more global work. Full-state VI is the sharper baseline. The same implementation meets its 10^{-10} update tolerance in 454 sweeps, or 4.20M billed backups, and the 10,000-sweep diagnostic has 1.76×10^{-9} error relative to the cached solution. On this instance, aggregation itself is not competitive; the experiment identifies behavior inside an approximation scheme, not a new best solver.

The preregistered endpoint failed. The original primary endpoint was $(2, 5)$ at 20 ms. It gave a residual-minus- geometric median of -0.0417 with 95% CI $[-0.0459, 0.0011]$, which is indeterminate under the equivalence rule. Worse, repeated executions changed the interval because 20 ms lies on a steep transient and selects a different iterate as machine load changes. I report rather than replace that outcome. The final-iterate and backup-indexed analyses are explicitly robustness analyses, motivated by this protocol failure, not retroactively preregistered endpoints.

5 What the negative result isolates

Endpoint matching is load-bearing. A second geometric arm makes the control failure concrete. The original slow schedule used 1,429 aggregate cycles, exactly the number induced by 10,000 iterations at cycle length seven. Transplanting it unchanged to $(1, 20)$ provides only 477 aggregate entries: it ends at $\varepsilon = .232$ and error 1.59. Recalibrating only its cycle count to 477 makes it end at $\varepsilon = .05024$ and error .209, back in the neighborhood of residual and fast geometric. The apparent failure was caused by an unfinished timetable, not by lack of feedback. More generally, an open-loop control must be matched in the clock on which adaptation occurs—here, aggregate entries, not total iterations.

Equivalence is not equality. At $(5, 2)$ the paired interval $[.006319, .006327]$ is exceptionally narrow and excludes zero. Calling the arms “indistinguishable” would therefore be false. The predeclared question was instead whether the entire interval lies inside a practically negligible $\pm.02$ region; it does. This distinction is important for negative results with low simulation variance: abundant precision can detect a difference too small to justify the adaptive mechanism. Conversely, the $(1, 20)$ interval crosses zero but is wider. Both satisfy the same practical criterion for different statistical reasons.

Artifact and audit trail. The repository records the frozen residual constants and timing endpoint before the residual arm was run, as well as the later schedule sweep, full configuration, software environment, per-seed endpoints, paired differences, and work-indexed curves. Compiled and pure-Python tests check Bellman backups, partitions, schedule semantics, seeded trajectories, and the $\ell_g = 1, \ell_a = 0$ reduction to ordinary VI. The reported sweep used CPython 3.14.6, NumPy 2.4.6, and Numba 0.67.0. Code and data will be available at `https://github.com/sunnysanitize/Residual-Epsilon-Aggregation`.

6 Limitations and conclusion

This study has one deterministic inventory instance, one residual statistic, one floor, and stochasticity only from representative sampling. The three update mixes and backup endpoints were added after the preregistered timing experiment. Billed backups omit residual computation and representation maintenance; charging them only strengthens VI but may change comparisons among aggregation schemes. Policy loss is quantized by the finite action set, and no claim is made about other MDPs, learned models, or settings where aggregation is genuinely faster than VI. In particular, feedback could matter when residuals are heterogeneous across regions, the useful floor is unknown, or the horizon ends before every schedule reaches it.

Within this scope, however, the result is stable: across three update mixes, Bellman-residual feedback never provides a practically meaningful improvement over a matched open-loop timetable. The broader methodological lesson is that adaptive algorithms require controls that reproduce the path induced by their feedback. Otherwise, ordinary scheduling can be mistaken for intelligence.

References

[1] Ward Whitt. Approximations of dynamic programs, I. *Mathematics of Operations Research*, 3(3):231–243, 1978.

[2] Dimitri P. Bertsekas and David A. Castanon. Adaptive aggregation methods for infinite horizon dynamic programming. *IEEE Transactions on Automatic Control*, 34(6):589–598, 1989.

[3] Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a unified theory of state abstraction for MDPs. In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics*, 2006.

[4] Guanting Chen, Johann Demetrio Gaebler, Matt Peng, Chunlin Sun, and Yinyu Ye. An adaptive state aggregation algorithm for Markov decision processes. *arXiv preprint arXiv:2107.11053*, 2021.

[5] Benjamin Van Roy. Performance loss bounds for approximate value iteration with state aggregation. *Mathematics of Operations Research*, 31(2):234–244, 2006.

[6] Satinder P. Singh, Tommi Jaakkola, and Michael I. Jordan. Reinforcement learning with soft state aggregation. In *Advances in Neural Information Processing Systems 7*, pages 361–368, 1995.

[7] Rémi Munos and Andrew Moore. Variable resolution discretization in optimal control. *Machine Learning*, 49(2–3):291–323, 2002.

[8] Orso Forghieri, Hind Castel, Emmanuel Hyon, and Erwan Le Pennec. Progressive state space disaggregation for infinite horizon dynamic programming. In *Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling*, pages 221–229, 2024.

[9] Ronald J. Williams and Leemon C. Baird III. Tight performance bounds on greedy policies based on imperfect value functions. Technical Report NU-CCS-93-14, Northeastern University, 1993.

[10] E. F. Arruda, M. D. Fragoso, and J. B. R. do Val. Approximate dynamic programming via direct search in the space of value function approximations. *European Journal of Operational Research*, 211(2):343–351, 2011.

[11] Andrew W. Moore and Christopher G. Atkeson. Prioritized sweeping: Reinforcement learning with less data and less real time. *Machine Learning*, 13(1):103–130, 1993.

[12] David Wingate and Kevin D. Seppi. Prioritization methods for accelerating MDP solvers. *Journal of Machine Learning Research*, 6:851–881, 2005.

[13] Matthieu Geist, Bilal Piot, and Olivier Pietquin. Is the Bellman residual a bad proxy? In *Advances in Neural Information Processing Systems 30*, pages 3205–3214, 2017.

[14] Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008.

[15] Olivier Guéant. Optimal market making. *Applied Mathematical Finance*, 24(2):112–154, 2017.